

Cricket Sports Application — Complete Full-Stack Project

I want you to build a production-style full-stack cricket sports web application.

The application should initially focus only on cricket, but the architecture must be designed so that other sports such as football, kabaddi, or other sports can potentially be added in the future.

Do not build a simple CRUD demo. Build a properly structured, scalable sports platform with a professional modern UI, a REST API backend, real-time live scoring, statistics, authentication, and an admin/scorer dashboard.

1. Technology Stack

Backend

Use:

- ASP.NET Core Web API
- C#
- Entity Framework Core
- MySQL database
- ASP.NET Core Identity
- JWT Authentication
- SignalR for real-time updates
- Swagger/OpenAPI
- Clean Architecture principles where appropriate
- Dependency Injection
- DTO pattern
- Repository pattern only where it provides actual value; do not over-engineer
- Global exception handling

- Logging
- Fluent validation or appropriate request validation
- Proper HTTP status codes

Frontend

Use:

- React
- TypeScript
- Vite
- React Router
- TanStack Query for API data fetching and caching
- Axios or a clean API client
- Tailwind CSS
- Framer Motion for normal UI animations
- Recharts for statistics and charts
- SignalR client for live updates

The frontend must be responsive and work well on:

- Desktop
- Tablet
- Mobile

Do not use 3D graphics, Three.js, Unity, Unreal Engine, or game engines.

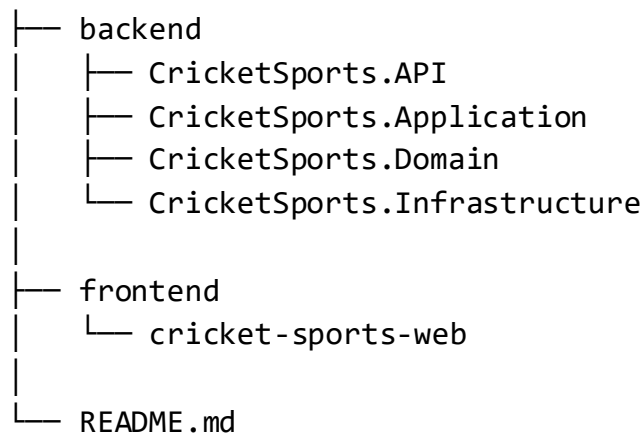
The visual design should be modern and similar in quality to a professional sports application, but it must have its own original design.

2. Project Structure

Create two main applications:

CricketSportsApp

|



The backend should have clear separation between:

Domain
Application
Infrastructure
API

Do not place all business logic directly inside controllers.

Controllers should remain thin.

Use services/application handlers for business logic.

3. Application Roles

The system should support the following roles:

Public User

Can:

- View live matches
- View upcoming matches
- View completed matches

- View scorecards
- View ball-by-ball commentary
- View teams
- View players
- View tournaments
- View rankings if implemented
- View news
- Register/login
- Mark favorite teams
- Mark favorite players

Admin

Can manage:

- Teams
- Players
- Tournaments
- Venues
- Matches
- News
- Banners or featured content
- Users if required

Scorer

The scorer should have access to live match scoring.

The scorer can:

- Start a match
- Start an innings
- Select batsmen
- Select bowler
- Enter each delivery result
- Record runs
- Record extras

- Record wickets
- End an over
- Change bowler
- End innings
- End match

Admin and scorer authorization must be protected using role-based authorization.

4. Main Public Application Pages

Create the following pages.

Home Page

The home page should contain:

Header

- Application logo
- Home
- Matches
- Teams
- Players
- Tournaments
- News
- Login/Profile

On mobile, create a responsive navigation menu.

Hero / Featured Area

Show:

- Featured live match
- Important tournament

- Featured cricket content

Live Matches Section

Show cards for currently live matches.

Each match card should display:

- Team 1
- Team 2
- Team logos
- Current score
- Overs
- Match status
- Live indicator

Example:

LIVE

INDIA

186/4

19.2 Overs

AUSTRALIA

Yet to bat

India lead by current match situation

Upcoming Matches

Display:

- Teams
- Match date
- Match time
- Venue
- Tournament

Recent Results

Display completed matches with:

- Final scores
- Winner
- Match result

Featured Tournament

Show a highlighted tournament.

Trending Players

Display player cards.

Latest Cricket News

Display news cards.

5. Matches Page

Create a Matches page with tabs:

Live

Upcoming

Completed

Users should be able to filter matches by:

- Tournament
- Date
- Team
- Match format

Match formats:

- Test
- ODI
- T20
- T10

Each match should navigate to a detailed Match Details page.

6. Match Details Page

This is one of the most important pages.

The match page should have a modern scoreboard at the top.

Example structure:

INDIA vs AUSTRALIA

LIVE • 2nd Innings

INDIA

186/4

19.2 Overs

Current Run Rate: 9.62

Target: 210

Need 24 runs from 4 balls

Below this, create tabs:

Live

Scorecard

Commentary

Statistics

Info

7. Live Match Tab

Display:

- Current score
- Overs
- Current batsmen
- Current bowler
- Batsmen runs and balls faced
- Bowler overs, runs, wickets
- Current run rate
- Required run rate
- Target
- Partnership
- Recent balls

Example:

Recent Balls

1 | 4 | 0 | w | 6 | 1

Use visual styling for:

- Four
- Six
- Wicket
- Dot ball
- Extras

Add simple UI animations.

For example:

When a FOUR is received through SignalR:

- Score smoothly updates
- Show a temporary animated overlay saying "FOUR!"
- Highlight the ball event

When a SIX occurs:

- Show "SIX!"
- Animate the score change

When a WICKET occurs:

- Show a wicket event animation
- Highlight the scorecard update

These should only be normal frontend animations using:

- CSS
- Tailwind
- Framer Motion

Do not use 3D animation.

8. Scorecard Tab

Create separate sections for:

Batting

Display:

- Player
- Dismissal
- Runs
- Balls
- Fours

- Sixes
- Strike Rate

Bowling

Display:

- Bowler
- Overs
- Maidens
- Runs
- Wickets
- Economy

Extras

Show:

- Wides
- No balls
- Byes
- Leg byes
- Total extras

Fall of Wickets

Example:

1-45
2-72
3-115

Support multiple innings.

9. Commentary Tab

Display ball-by-ball commentary.

Example:

19.1

FOUR!

Excellent cover drive through the off side.

19.2

1 RUN

19.3

WICKET!

Caught at deep mid-wicket.

Each commentary event should contain:

- Over number
- Ball number
- Event type
- Runs
- Description

Allow commentary to update in real time using SignalR.

10. Statistics Tab

Display useful match statistics.

Include:

- Run rate comparison
- Team score progression

- Runs per over
- Wickets progression
- Partnership statistics

Use Recharts.

Charts must be responsive.

11. Teams Module

Create a Teams page.

Display all teams.

Each team card should show:

- Team logo
- Team name
- Country/region

Create a Team Details page with:

Overview

Squad

Matches

Results

Statistics

The squad should display player cards.

12. Players Module

Create a Players page.

Allow:

- Search
- Filtering
- Sorting

Player profile should contain:

Profile
Career Statistics
Recent Matches
Records

Player information:

- Name
- Profile image
- Team
- Role
- Batting style
- Bowling style

Career statistics should support:

- Matches
- Runs
- Highest score
- Batting average
- Strike rate
- Hundreds
- Fifties
- Wickets
- Bowling average
- Economy rate

Design the player profile professionally.

13. Tournaments Module

Create tournament pages.

A tournament should contain:

Overview

Matches

Teams

Points Table

Top Players

Statistics

The points table should display:

- Position
- Team
- Matches played
- Wins
- Losses
- Points
- Net Run Rate

Support tournaments such as:

- IPL
- International series
- World Cup
- Domestic tournaments

However, do not hard-code tournament names.

Everything should come from the database.

14. News Module

Create a cricket news section.

Admin should be able to:

- Create news
- Edit news
- Delete news
- Upload or attach image URL
- Mark news as featured

Public users should be able to:

- View news list
- Open full news article
- See related news

Create a clean article page.

15. Authentication

Implement:

- Register
- Login
- Logout
- JWT authentication
- Refresh token strategy if appropriate
- Role-based authorization

Roles:

User

Admin

Scorer

Do not expose admin or scorer functionality to unauthorized users.

16. Admin Dashboard

Create a separate admin dashboard.

Dashboard should show summary cards:

- Total matches
- Live matches
- Total players
- Total teams
- Total tournaments

Create management pages for:

Teams

Players

Tournaments

Venues

Matches

News

Users

Each module should support appropriate CRUD operations.

Use:

- Search
- Pagination
- Validation
- Confirmation before destructive actions

17. Live Scoring Dashboard

This is a major feature.

Create a dedicated scorer interface.

The scorer should be able to record each delivery.

Interface example:

Current Score: 145/3

Overs: 16.2

Striker: Player A

Non-Striker: Player B

Bowler: Player C

Select Event:

0

1

2

3

4

6

Wide

No Ball

Bye

Leg Bye

Wicket

Undo Last Ball

When a delivery is submitted:

1. Validate the match state.

2. Store the delivery in the database.
3. Update the innings score.
4. Update batsman statistics.
5. Update bowler statistics.
6. Update overs.
7. Rotate strike where appropriate.
8. Handle over completion.
9. Calculate run rate.
10. Update partnerships.
11. Record wicket information if applicable.
12. Broadcast the updated state through SignalR.

Do not trust the frontend to calculate official match results.

All important cricket scoring logic must be calculated on the backend.

18. Cricket Scoring Engine

Design the scoring engine carefully.

Create a proper Delivery entity.

A delivery should contain information such as:

Id

MatchId

InningsId

OverNumber

BallNumber

StrikerId

NonStrikerId

BowlerId

RunsOffBat

ExtraRuns

ExtraType

IsWicket
WicketType
DismissedPlayerId

Timestamp
Commentary

Extra types:

Wide
NoBall
Bye
LegBye

Wicket types should be extensible.

The scoring engine should correctly handle:

- Legal deliveries
- Illegal deliveries
- Wide balls
- No balls
- Byes
- Leg byes
- Wickets
- Strike rotation
- End of over
- New batsman
- New bowler
- Innings completion
- Match completion

Design this logic so it can later be expanded for more advanced cricket rules.

19. Database Entities

Initially include entities similar to:

ApplicationUser

Team

Player

Venue

Tournament

TournamentTeam

Match

MatchTeam

Innings

Delivery

BattingScorecard

BowlingScorecard

Partnership

FallOfWicket

NewsArticle

FavoriteTeam

FavoritePlayer

Use proper foreign key relationships.

Avoid unnecessary duplication.

Add indexes where useful.

Create EF Core migrations.

20. SignalR Real-Time System

Create a SignalR Hub for live match updates.

The flow should be:

```
Scorer
  ↓
POST /api/scoring/delivery
  ↓
Backend validates and saves delivery
  ↓
Database updated
  ↓
SignalR Hub broadcasts event
  ↓
Connected clients update immediately
```

Users watching a particular match should receive updates only for that match.

Use groups based on MatchId.

Example concept:

match-`{matchId}`

Events should support:

- Score updated
- New ball
- Wicket
- Innings completed
- Match completed

Ensure events have sequence information or another mechanism to help clients avoid displaying updates out of order.

21. API Design

Create RESTful APIs.

Example:

`/api/auth`

`/api/matches`

`/api/matches/{id}`

`/api/matches/{id}/live`

`/api/matches/{id}/scorecard`

`/api/matches/{id}/commentary`

`/api/matches/{id}/statistics`

`/api/teams`

`/api/teams/{id}`

`/api/players`

`/api/players/{id}`

`/api/tournaments`

`/api/tournaments/{id}`

`/api/news`

Admin:

`/api/admin/teams`

`/api/admin/players`

`/api/admin/tournaments`

`/api/admin/venues`

`/api/admin/matches`

`/api/admin/news`

Scoring:

```
/api/scoring/matches/{matchId}/start
/api/scoring/matches/{matchId}/innings/start
/api/scoring/matches/{matchId}/delivery
/api/scoring/matches/{matchId}/undo
/api/scoring/matches/{matchId}/end-innings
/api/scoring/matches/{matchId}/end-match
```

Use DTOs for requests and responses.

Do not expose EF Core entities directly.

22. Frontend Architecture

Organize the React application clearly.

Suggested structure:

```
src
|
├── api
├── components
|   ├── common
|   ├── match
|   ├── player
|   ├── team
|   └── tournament
├── features
|   ├── auth
|   ├── matches
|   ├── teams
|   ├── players
|   ├── tournaments
|   ├── news
|   └── scoring
└──
```

- |— hooks
- |— layouts
- |— pages
- |— routes
- |— types
- |— utils
- |— signalr

Use TypeScript interfaces/types properly.

Do not use excessive any.

23. UI Design Requirements

The application should look like a premium modern sports platform.

Requirements:

- Clean typography
- Strong match cards
- Clear live indicators
- Responsive layout
- Good spacing
- Professional scorecards
- Smooth transitions
- Skeleton loaders
- Empty states
- Error states
- Loading states

Use normal animations only.

Examples:

- Score number transitions
- FOUR overlay

- SIX overlay
- WICKET overlay
- Match card hover effects
- Tab transitions
- Page transitions

Do not add unnecessary flashy animations.

The UI should prioritize readability of cricket information.

Do not implement all of these now.

But avoid architectural decisions that would make them impossible later.

25. Development Approach

Do not generate the entire project as one giant unstructured block.

Build the project in phases.

Phase 1

Set up:

- Backend solution
- Clean architecture
- MySQL connection
- EF Core
- Identity
- JWT
- Swagger
- Global exception handling

Phase 2

Create:

- Teams
- Players
- Tournaments
- Venues
- Matches

Implement migrations and CRUD APIs.

Phase 3

Build the React frontend:

- Layout
- Navigation
- Home page
- Matches
- Teams
- Players
- Tournaments

Connect frontend to APIs.

Phase 4

Build Match Details:

- Live view
- Scorecard
- Commentary
- Statistics

Phase 5

Build Admin Dashboard.

Phase 6

Build the Live Scoring Engine.

Test cricket scoring logic carefully.

Phase 7

Add SignalR real-time updates.

Phase 8

Add frontend visual event animations for:

- FOUR
- SIX
- WICKET
- Match completion

Phase 9

Improve:

- Validation
- Error handling
- Loading states
- Responsive design
- Security
- Performance

26. Important Coding Requirements

Before generating major sections:

1. Inspect the existing project structure.
2. Do not overwrite working files unnecessarily.
3. Reuse existing patterns where appropriate.
4. Explain what will be created before making large architectural changes.
5. Generate code that compiles.
6. Avoid placeholder implementations for core business logic.
7. Do not hard-code teams, players, or matches in production code.
8. Use database seed data only for development/testing.
9. Keep backend business logic separate from frontend logic.
10. Test API endpoints after implementation.
11. Fix compilation errors before moving to the next phase.
12. Document setup instructions in README.md.

27. First Task to Start With

Start by analyzing the current repository.

Then:

1. Show the proposed folder structure.
2. Identify what already exists.
3. Identify missing dependencies.
4. Create or modify only the backend foundation first.
5. Set up ASP.NET Core Clean Architecture.
6. Configure MySQL with EF Core.
7. Configure Identity and JWT authentication.
8. Configure Swagger.
9. Add global exception handling.
10. Ensure the project builds successfully.

Do not implement the entire application in the first step.

After the foundation is complete and compiling, continue module by module in the planned phases.